AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

YOUR FULL NAME, Your Institute / Department, India

[FILL IN – 150–250 words.] Summarize the problem (rural water conservation and pond-site selection), the approach (geospatial + rainfall data analysis exposed through a web application), the key system components you built, and the main results (e.g., number of candidate sites evaluated, accuracy/usefulness of the recommended pond depth and storage capacity, and any performance figures for the system itself).

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

ACM Reference Format:
Your Full Name. 2026. AI-based Village Pond Planning System: CSD Assignment 1 – Final Technical Report. 1, 1 (September 2026), ?? pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

[MUST BE INCLUDED] Introduce the problem of rural water conservation and why pond-based rainwater harvesting is an effective solution. State the goal of the assignment (an AI/geospatial-assisted web application that recommends suitable pond locations) and give a short roadmap of the rest of the report.

1.1 Motivation

[FILL IN] Explain, in your own words, why manual site selection is hard (terrain complexity, lack of centralized rainfall data, unavailability of land-ownership records, etc.) and why an automated tool helps village administrators.

1.2 Scope of the Project

[MUST BE INCLUDED] List, in one paragraph or a short bullet list, what your system does and does *not* cover (e.g., "estimates catchment area and expected runoff for a user-selected point; does not perform structural/civil engineering design of the pond embankment").

2 Problem Statement and Requirements

[MUST BE INCLUDED] Restate the assignment's functional requirements in your own words and map each to the module of your system that satisfies it. A table works well here.

Author's Contact Information: Your Full Name, Your Institute / Department, Your City, India, your.email@institute.ac.in.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to CSD ASsignemnt

Table 1. Functional requirements and where they are implemented

Requirement	Implemented in
Satellite imagery display	*[module/file]*
Contour map visualization	*[module/file]*
Available-land identification	*[module/file]*
Catchment area estimation	*[module/file]*
Historical rainfall query	*[module/file]*
Runoff volume estimation	*[module/file]*
Pond depth / storage recommendation	*[module/file]*
Combined overlay / results view	*[module/file]*

2.1 Non-Functional Requirements

[MUST BE INCLUDED] State performance, scalability, availability, usability, and security requirements you set for yourself, e.g. response time for a query, number of concurrent users the system should tolerate, expected uptime, browser support, etc.

3 System Architecture and High-Level Design

[MUST BE INCLUDED] Describe the overall architecture: client, server, database, and external API layers. Include an architecture diagram (Figure **??**) showing how data flows from the map UI to the backend, to the elevation/rainfall APIs, to the database, and back.

Fig. 1. High-level system architecture.

3.1 Technology Stack

[MUST BE INCLUDED] List the actual stack you used (language, web framework, database, frontend library/framework, mapping library, and which elevation/rainfall APIs you queried). Justify any deviation from the suggested stack in the assignment brief.

4 Methodology

[MUST BE INCLUDED] This section should explain the actual algorithms and data pipelines you implemented – this is the core of the "Terrain and Catchment Analysis" evaluation criterion, so be specific and quantitative.

4.1 Terrain and Elevation Analysis

[FILL IN] Explain how you obtained and processed the Digital Elevation Model (DEM) / elevation API data, how contour lines were generated, and how you used OpenCV (or your chosen library) for any image/terrain processing.

4.2 Catchment Area Delineation

[FILL IN] Explain the algorithm used to estimate the catchment area contributing runoff to a candidate point (e.g., flow-direction/flow-accumulation from the DEM, or a simplified buffer/slope-based heuristic). State assumptions and limitations.

4.3 Rainfall Data Integration

[FILL IN] Describe which rainfall API(s) you queried (IMD, Open-Meteo, NASA POWER, etc.), the time range of historical data used, and how you aggregated it into annual/seasonal statistics.

5 Implementation

5.1 Backend and API Design

[MUST BE INCLUDED] List your REST/GraphQL endpoints (a table of `METHOD /path -- purpose` works well), authentication approach if any, and how errors from third-party APIs are handled.

5.2 Frontend and Visualization

[MUST BE INCLUDED] Describe the map interface, the visualization library used, and how the results overlay (site, catchment boundary, rainfall stats, runoff volume, pond dimensions) is presented to the user. Include a screenshot.

Fig. 2. Application interface showing the results overlay.

5.3 Database and Storage

[FILL IN] Explain how spatial data, rainfall records, and computed recommendations are persisted and queried.

6 CSD Themes and Topics Applied in the Project

[MUST BE INCLUDED] This section is where you explicitly connect your implementation back to core Computer System Design (CSD) concepts taught in the course. For every theme you list, name the concrete component of your system where it appears and justify why you made that design choice (not just that the feature exists). Add or remove rows/subsections to match what you actually built – do not claim a theme you did not implement.

[FILL IN] Add a short paragraph after the table walking through the two or three themes you consider most significant to this project and explaining, in more depth, the specific engineering decision behind each (e.g., "We added a load balancer because X" or "We chose not to implement microservices because Y").

Table 2. Mapping of core CSD themes/topics to their use in this project

CSD Theme/Topic	Where used in the project	Justification / design rationale
API Design (REST)	*[e.g., `/api/catchment`, `/api/rainfall`]*	*[Why REST, versioning, request/response contract]*
Load Balancing	*[e.g., Nginx in front of N app instances]*	*[Why needed – concurrent map queries, API rate limits, etc.]*
Caching	*[e.g., cached rainfall API responses / tile cache]*	*[Reduces external API calls, improves latency]*
Database Indexing / Query Optimization	*[e.g., spatial index on site coordinates]*	*[Speeds up geospatial queries]*
Concurrency / Asynchronous Processing	*[e.g., async calls to elevation + rainfall APIs]*	*[Avoids blocking the UI while multiple APIs are queried]*
Microservices vs. Monolith	*[state which you chose]*	*[Trade-offs you considered]*
Design Patterns	*[e.g., Factory for API clients, Strategy for runoff models]*	*[Why the pattern fit the problem]*
Authentication / Authorization	*[if applicable]*	*[Who can trigger analyses / view admin data]*
Containerization / Deployment	*[e.g., Docker, docker-compose]*	*[Reproducible environment, ease of grading/demo]*
Error Handling and Resilience	*[e.g., retries/fallbacks when a rainfall API is down]*	*[System should degrade gracefully]*
Algorithms and Complexity	*[e.g., flow-accumulation algorithm complexity]*	*[Why this algorithm was chosen over alternatives]*
Testing Strategy	*[unit/integration/system tests – see Section **??**]*	*[What confidence this gives in correctness]*
Version Control / CI-CD	*[e.g., GitHub Actions pipeline]*	*[How it kept the team in sync / caught regressions]*

7 Results and Evaluation

[MUST BE INCLUDED] Present representative results: example village/site with its computed catchment area, rainfall statistics, estimated runoff, and recommended pond dimensions. Include maps or charts (Figure **??**) and, if possible, compare your estimated values against any ground-truth or published reference figures.

Fig. 3. Example results overlay for a selected village.

7.1 Performance

[MUST BE INCLUDED] Report response times for key operations (e.g., time to fetch rainfall data, time to compute catchment area for a given site) and how the system behaves under load if you tested that.

8 Discussion and Limitations

[FILL IN] Reflect honestly on what worked well, what did not, data-quality issues (e.g., coarse DEM resolution, gaps in rainfall records), and any simplifying assumptions that affect the accuracy of the recommendations.

9 AI Tool Usage Declaration

[MUST BE INCLUDED – required by the assignment's LLM Usage Policy.] State which AI tools (ChatGPT, Claude, Gemini, GitHub Copilot, etc.) were used, for which specific tasks (brainstorming, debugging, documentation, refactoring, etc.) like AI was used to reformat this tempalte, and confirm that all AI-assisted code was reviewed, understood, and modified by the team before submission. Do not leave this section blank.

A Source Code and Repository

[MUST BE INCLUDED] Provide the repository link and a short note on its folder structure.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.